Customer Churn Prediction & Agentic Retention Strategy

From Predictive Analytics to Intelligent Intervention

GenAI Capstone Project — Project 5

Combined Report: Milestone 1 & Milestone 2

Name	Enrolment No.
Gokul VKS	2401020094
Shreyash Golhani	2401020069
Vaageesh Kumar Singh	2401020073
Mohammad Affan Anas	2401010280

Live Demo: genaicapstoneproject.streamlit.app
GitHub: github.com/Shreyashgol/genAI_capstone_project
Dataset: Telco Customer Churn (Kaggle / IBM Sample)

Submitted in partial fulfilment of the GenAI Capstone Course Requirements

April 2026

Abstract

Customer churn is one of the most critical operational challenges in the telecommunications industry. This report presents the complete design, implementation, and evaluation of a two-milestone AI-driven customer analytics system. **Milestone 1** applies classical supervised machine learning — Logistic Regression, Decision Tree, and Random Forest — to the publicly available Telco Customer Churn dataset, covering end-to-end preprocessing, SMOTE-based class balancing, model training, and comparative evaluation. **Milestone 2** extends this into a fully agentic AI retention strategy assistant powered by a LangGraph orchestration framework. The agent autonomously assesses churn risk, retrieves evidence-grounded retention strategies from a domain-specific knowledge base via Retrieval-Augmented Generation (RAG) using Chroma and `intfloat/e5-small` embeddings, plans personalised interventions using the Groq-hosted Llama 3.1 8B model, and generates structured retention reports with cited sources and ethical disclosures. The system is deployed as a publicly accessible Streamlit application, demonstrating full-stack AI engineering from predictive ML to agentic reasoning.

Contents

1 Introduction **3**
- 1.1 Business Problem . . . 3
- 1.2 Problem Statement . . . 3
- 1.3 Project Scope and Milestones . . . 3

2 Dataset Description **3**
- 2.1 Source . . . 3
- 2.2 Structure and Features . . . 3
- 2.3 Class Distribution . . . 4

3 Milestone 1: ML-Based Churn Prediction **4**
- 3.1 System Architecture . . . 4
- 3.2 Data Preprocessing . . . 4
  - 3.2.1 Type Correction . . . 4
  - 3.2.2 Column Removal . . . 4
  - 3.2.3 Categorical Encoding . . . 5
- 3.3 Feature Engineering . . . 5
- 3.4 Class Imbalance Handling — SMOTE . . . 5
- 3.5 Train/Test Split and Scaling . . . 5
- 3.6 Models . . . 5
  - 3.6.1 Logistic Regression . . . 5
  - 3.6.2 Decision Tree Classifier . . . 5
  - 3.6.3 Random Forest Classifier . . . 6
- 3.7 Evaluation . . . 6
  - 3.7.1 Metrics . . . 6
  - 3.7.2 Results . . . 6
  - 3.7.3 Key Observations . . . 6
- 3.8 Feature Importance Analysis . . . 6

4 Milestone 2: Agentic AI Retention Strategy Assistant **7**
- 4.1 Overview and Motivation . . . 7

4.2 System Architecture . . . 7
4.3 LangGraph State Management . . . 7
4.4 Node 1: Risk Assessment (`assess_risk`) . . . 8
4.5 Node 2: RAG Retrieval (`retrieve_strategies`) . . . 8
4.5.1 Knowledge Base . . . 8
4.5.2 Embedding and Vector Store . . . 8
4.5.3 Retrieval Query Construction . . . 9
4.6 Node 3: Intervention Planning (`plan_intervention`) . . . 9
4.6.1 LLM Configuration . . . 9
4.6.2 Prompting Strategy and Hallucination Mitigation . . . 9
4.7 Node 4: Report Generation (`generate_report`) . . . 10
4.7.1 Ethical Disclaimer . . . 10
4.8 Graph Compilation . . . 10

5 Application Design and Deployment **11**
5.1 User Interface . . . 11
5.2 Deployment . . . 11

6 Technology Stack **12**

7 Project Structure **12**

8 Ethical Considerations **13**
8.1 Bias and Fairness . . . 13
8.2 Data Privacy . . . 13
8.3 Model Transparency . . . 13
8.4 AI-Generated Recommendations . . . 13
8.5 Responsible Use . . . 14

9 Limitations and Future Work **14**
9.1 Current Limitations . . . 14
9.2 Future Enhancements . . . 14

10 Conclusion **14**

1 Introduction

1.1 Business Problem

Customer churn — the event of a customer discontinuing their subscription — represents a substantial financial liability for telecommunications companies. Industry research consistently indicates that acquiring a new customer costs five to seven times more than retaining an existing one. Proactive identification of at-risk customers enables targeted retention interventions such as personalised discounts, contract renegotiations, and proactive support outreach, resulting in significantly higher return on investment compared to reactive strategies.

1.2 Problem Statement

Given a customer's demographic profile, account configuration, and service subscription patterns, the system must: (1) predict whether the customer is likely to churn with an associated probability score; (2) identify the key features driving that risk; and (3) autonomously generate a structured, evidence-grounded retention strategy tailored to the customer's specific profile.

1.3 Project Scope and Milestones

This project is structured in two progressive milestones:

- **Milestone 1 (Mid-Semester):** Classical ML pipeline for churn prediction, including EDA, preprocessing, feature engineering, SMOTE balancing, model training, evaluation, and deployment of an interactive Streamlit UI.
- **Milestone 2 (End-Semester):** Extension into a LangGraph-based agentic AI system with RAG retrieval, LLM-powered intervention planning, structured report generation, and ethical disclosures.

2 Dataset Description

2.1 Source

The dataset used is the Telco Customer Churn dataset, originally released by IBM as a sample dataset and distributed via Kaggle (`blastchar/telco-customer-churn`). It contains historical records for 7,043 customers of a fictional US telecommunications company.

2.2 Structure and Features

The raw dataset comprises 7,043 rows and 21 columns. The target variable is `Churn` (binary: Yes/No). The remaining 20 columns serve as predictive features, grouped as follows:

Table 1: Dataset Feature Summary

Category	Type	Features
Identity	Categorical (ID)	`customerID`
Demographics	Categorical/Binary	`gender`, `SeniorCitizen`, `Partner`, `Dependents`
Account Info	Mixed	`tenure`, `Contract`, `PaperlessBilling`, `PaymentMethod`
Charges	Continuous	`MonthlyCharges`, `TotalCharges`
Services	Categorical	`PhoneService`, `MultipleLines`, `InternetService`, `OnlineSecurity`, `OnlineBackup`, `DeviceProtection`, `TechSupport`, `StreamingTV`, `StreamingMovies`
Target	Binary	`Churn`

2.3 Class Distribution

The dataset exhibits a notable class imbalance: approximately 73.5% of customers did not churn (`No`) while 26.5% churned (`Yes`). This imbalance was addressed during preprocessing using SMOTE oversampling applied exclusively to the training partition to prevent data leakage.

3 Milestone 1: ML-Based Churn Prediction

3.1 System Architecture

The Milestone 1 pipeline follows a standard supervised ML architecture with a deployment layer:

```
Raw CSV → Preprocessing → Feature Engineering → Encoding & Scaling
→ Train/Test Split (80/20) → SMOTE → Model Training (LR / DT / RF)
→ Evaluation → Serialisation (.pkl) → Streamlit UI → Deployment
```

3.2 Data Preprocessing

3.2.1 Type Correction

The `TotalCharges` column was stored as a string due to whitespace entries for customers with zero tenure. These were coerced to numeric using `pd.to_numeric(..., errors='coerce')` and resulting `NaN` values were imputed with zero — semantically appropriate since new customers have no accumulated charges.

3.2.2 Column Removal

The `customerID` column is a unique identifier with no predictive value and was dropped prior to modelling.

3.2.3 Categorical Encoding

The binary target variable `Churn` was mapped from {No, Yes} to {0, 1}. All remaining categorical features were one-hot encoded using `pandas.get_dummies(drop_first=True)` to avoid multicollinearity.

3.3 Feature Engineering

Two new features were engineered to enrich the predictive representation:

1. **TenureGroup**: Customer tenure (months) was discretised into four interpretable bands: 0–1 Year ($\leq$12 months), 1–2 Years (13–24), 2–4 Years (25–48), Over 4 Years (>48). This captures non-linear tenure effects.
2. **ChargeRatio**: Defined as $\frac{\texttt{TotalCharges}}{\texttt{MonthlyCharges}+1}$, this feature acts as a proxy for effective customer lifetime value by capturing the relationship between cumulative and periodic billing. Adding 1 prevents division by zero for new customers.

3.4 Class Imbalance Handling — SMOTE

The original 73.5%/26.5% split would bias models toward predicting the majority class (No churn). SMOTE (Synthetic Minority Oversampling Technique) [3] was applied exclusively on the training partition after the train/test split. Applying SMOTE only to training data is critical to prevent data leakage — the test set must reflect the true real-world distribution.

SMOTE generates synthetic minority class samples by interpolating between existing minority instances in feature space, producing a balanced training set without duplicating real records.

3.5 Train/Test Split and Scaling

The processed dataset was split 80%/20% using `train_test_split` with `random_state=42`. After SMOTE resampling, `StandardScaler` was applied (fit on training data only, applied to both partitions) to normalise feature scales, which is particularly important for Logistic Regression.

3.6 Models

3.6.1 Logistic Regression

A linear probabilistic classifier modelling the log-odds of churn as a linear combination of input features. Valued for interpretability — coefficients directly reveal each feature's directional influence. Trained on scaled, SMOTE-balanced training data with `max_iter=1000`.

3.6.2 Decision Tree Classifier

Recursively partitions the feature space using axis-aligned splits maximising class purity (Gini impurity). Maximum tree depth restricted to 5 (`max_depth=5`) to prevent overfitting while maintaining interpretable decision rules suitable for compliance and auditability.

3.6.3 Random Forest Classifier

An ensemble of 100 decorrelated decision trees (`n_estimators=100, max_depth=10`), each trained on a bootstrapped sample with a random feature subset at each split. Averaging predictions across trees reduces variance and consistently achieves higher generalisation performance. Feature importance scores are extracted as mean impurity decrease across the ensemble.

3.7 Evaluation

3.7.1 Metrics

Four standard binary classification metrics were used:

- **Accuracy**: Fraction of correct predictions over all samples.
- **Precision**: Of all predicted churners, the fraction that actually churned. High precision reduces false alarms.
- **Recall**: Of all actual churners, the fraction correctly identified. High recall minimises missed at-risk customers — particularly critical in churn prediction where missing a churner is costly.
- **F1-Score**: Harmonic mean of Precision and Recall, providing a single balanced metric.

3.7.2 Results

Table 2: Model Performance on Held-Out Test Set (20% split, random_state=42)

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	79.35%	59.67%	67.83%	63.49%
Decision Tree	77.15%	55.24%	72.12%	62.56%
Random Forest	79.28%	59.06%	70.78%	64.39%

3.7.3 Key Observations

1. **Random Forest** achieved the highest F1-Score (64.39%) and competitive recall, demonstrating the advantage of ensemble averaging over single models.
2. **Logistic Regression** performed competitively given its simplicity, remaining the most interpretable option for explaining churn drivers to non-technical stakeholders.
3. **Decision Tree** showed the highest recall (72.12%), making it preferable in scenarios where minimising missed churners is prioritised over precision.
4. The moderate recall across all models reflects the inherent difficulty of the churn prediction task given the original class distribution.

3.8 Feature Importance Analysis

Random Forest feature importance analysis consistently identifies the following top churn drivers (ranked by mean impurity decrease):

1. **Tenure / TenureGroup**: Shorter-tenure customers exhibit significantly higher churn rates.
2. **Contract Type**: Month-to-month contracts strongly associate with churn; annual and two-year contracts show substantially lower rates.
3. **MonthlyCharges**: Higher monthly bills correlate with increased churn probability.
4. **TechSupport**: Absence of technical support subscription is a strong predictor.
5. **InternetService**: Fibre optic customers churn at a higher rate than DSL customers.
6. **ChargeRatio**: The engineered feature provides additional predictive signal beyond the base charge columns, validating the feature engineering step.

These findings align with telecommunications domain knowledge and provide actionable business intelligence.

4 Milestone 2: Agentic AI Retention Strategy Assistant

4.1 Overview and Motivation

Milestone 1 answers *who will churn*. Milestone 2 answers *what to do about it*. The system is extended from a passive predictor into an autonomous retention strategist that reasons about risk, retrieves evidence-based knowledge, plans personalised interventions, and generates structured reports — all within a single orchestrated agentic pipeline.

4.2 System Architecture

The Milestone 2 agentic pipeline is built on LangGraph's `StateGraph` abstraction. The complete flow is as follows:

```
Customer Input → ML Model (M1) → ChurnAgentState
↓
Node 1:  assess_risk (risk tier + driver extraction)
↓
Node 2:  retrieve_strategies (RAG: Chroma + e5-small)
↓
Node 3:  plan_intervention (Groq LLM: Llama 3.1 8B)
↓
Node 4:  generate_report (JSON parse + sources + disclaimer)
↓
Structured Retention Report → Streamlit UI
```

4.3 LangGraph State Management

The agent's shared memory is defined as a `TypedDict` that flows through all nodes:

```
class ChurnAgentState(TypedDict):
    customer_data: dict         # Raw customer feature dict from
        Streamlit form
    churn_probability: float    # Output from Milestone 1 ML model (0.0
        to 1.0)
    risk_level: str             # Derived: "low", "medium", or "high"
```

```
    risk_drivers: list          # Key features contributing to churn risk
    retrieval_query: str        # Semantic query sent to Chroma vector
        store
    retrieved_strategies: list # Text chunks retrieved from knowledge
        base
    llm_reasoning: str          # Raw LLM JSON output from
        plan_intervention
    retention_report: dict      # Final structured report dict
    error: Optional[str]        # Populated if any node fails gracefully
```

Listing 1: ChurnAgentState definition (state.py)

Explicit state management ensures that every step's inputs and outputs are traceable, testable, and auditable — a key requirement of the project rubric.

4.4 Node 1: Risk Assessment (assess_risk)

This node classifies the churn probability from the Milestone 1 model into a risk tier and extracts human-readable risk drivers from the raw customer feature values.

Risk tier classification:

- **High Risk**: churn probability $\geq 65\%$
- **Medium Risk**: $35\% \leq$ probability $< 65\%$
- **Low Risk**: probability $< 35\%$

Driver extraction logic maps raw feature values to interpretable signals, including: month-to-month contract, short tenure (<12 months), fibre optic internet, no online security, no tech support, electronic check payment, high monthly charges (>$70), senior citizen status, and no partner/dependents (lower switching cost). This deterministic extraction ensures the subsequent LLM receives a pre-reasoned, grounded profile rather than raw numbers.

4.5 Node 2: RAG Retrieval (retrieve_strategies)

4.5.1 Knowledge Base

Seven industry-sourced PDFs form the retrieval corpus, covering customer revenue research, customer success management, strategic retention research, loyalty and satisfaction frameworks, and customer 360 playbooks. Documents were chunked at 400 tokens with 50-token overlap using `RecursiveCharacterTextSplitter`, producing approximately 2,000 retrievable chunks.

4.5.2 Embedding and Vector Store

Chunks were embedded using `intfloat/e5-small` via the `langchain-huggingface` integration and persisted to a local Chroma vector store (`chroma_db/`). The vector store is committed to the repository, eliminating re-embedding costs on deployment and ensuring RAG is available immediately on startup.

4.5.3 Retrieval Query Construction

Rather than passing raw features to the retriever, Node 2 constructs a semantically rich query incorporating the risk tier, contract type, tenure, internet service, and the top two extracted risk drivers. This targeted query consistently retrieves more relevant chunks than a generic customer description.

```
query = (
    f"customer retention strategy for {risk_level} churn risk customer "
    f"with {contract.lower()} contract, {tenure} months tenure"
)
if internet:
    query += f", {internet.lower()} internet service"
if drivers:
    query += f". Key issues: {', '.join(drivers[:2])}"
```

Listing 2: Retrieval query construction (nodes.py)

The top 4 most similar chunks are retrieved via cosine similarity search (`k=4`).

4.6 Node 3: Intervention Planning (`plan_intervention`)

4.6.1 LLM Configuration

The Groq API hosts `llama-3.1-8b-instant` at free-tier rates with very low latency. The model is called with `temperature=0.3` to favour consistent, factual outputs over creative variation.

4.6.2 Prompting Strategy and Hallucination Mitigation

The system prompt enforces five strict rules to reduce hallucination:

```
RETENTION_SYSTEM_PROMPT = """You are an expert customer retention
   strategist.

Rules you must follow:
1. Be specific - reference the customer's actual situation.
2. Be actionable - every recommendation must be executable today.
3. Be honest - if churn risk is low, say so and recommend lighter
   actions.
4. Avoid hallucinating offers not grounded in the retrieved knowledge.
5. Respond with a single valid JSON object and nothing else.

The JSON must have exactly these keys:
- "risk_summary": string, 2-3 sentences explaining why this customer is
   at risk
- "recommended_actions": list of 3-5 strings, each a concrete retention
   action
- "reasoning": string, 2-3 sentences explaining why these actions fit
"""
```

Listing 3: System prompt (prompts.py)

The user prompt injects the full customer profile, risk tier, extracted drivers, and all retrieved knowledge chunks, explicitly instructing the model to ground recommendations in the retrieved context and not to invent policies.

4.7 Node 4: Report Generation (generate_report)

This node parses the LLM's JSON output with two-level fallback robustness:

1. **Direct parse**: Attempt `json.loads()` on the raw LLM output.
2. **Fence stripping**: Strip markdown code fences (```` ```json ```) and retry.
3. **Heuristic extraction**: Find the first { and last } and parse the substring.
4. **Safe fallback**: If all parsing fails, return a structured report with standard best-practice actions and a note that structured parsing failed.

The final report always contains: risk summary, recommended actions, agent reasoning, knowledge base source previews (first 120 characters of each retrieved chunk), and the ethical disclaimer.

4.7.1 Ethical Disclaimer

Every generated report includes a mandatory hardcoded disclaimer:

> *This retention strategy was generated by an AI system and is intended as decision-support for human agents, not as an autonomous decision-making tool. Recommendations should be reviewed by a qualified customer success professional before any action is taken. Customer data used in this analysis must be handled in accordance with applicable data protection regulations (e.g., GDPR, CCPA). The probability score is a statistical estimate and does not guarantee future customer behaviour.*

4.8 Graph Compilation

```
workflow = StateGraph(ChurnAgentState)

workflow.add_node("assess_risk", assess_risk)
workflow.add_node("retrieve_strategies", retrieve_strategies)
workflow.add_node("plan_intervention", plan_intervention)
workflow.add_node("generate_report", generate_report)

workflow.set_entry_point("assess_risk")
workflow.add_edge("assess_risk", "retrieve_strategies")
workflow.add_edge("retrieve_strategies", "plan_intervention")
workflow.add_conditional_edges(
    "plan_intervention",
    _should_continue_after_planning,
    {"generate_report": "generate_report"}
)
workflow.add_edge("generate_report", END)

retention_agent = workflow.compile()
```

Listing 4: LangGraph StateGraph compilation (graph.py)

The conditional edge after `plan_intervention` ensures the pipeline always reaches `generate_report` even if the LLM call fails, producing a safe fallback report in all cases.

5 Application Design and Deployment

5.1 User Interface

The Streamlit application provides a five-page interactive interface navigated via session state:

1. **Intro Page**: Project overview, key features (multiple models, real-time analysis, AI retention), and entry point.
2. **Customer Input**: A 19-field form covering demographics, phone services, internet services, additional services, and billing information.
3. **Model Selection**: Choice between Logistic Regression, Decision Tree, and Random Forest with brief descriptions of each model's characteristics.
4. **Results Page**: Churn probability gauge (Plotly), probability breakdown cards, top-10 feature importance bar chart, model performance metrics (Accuracy/Precision/Recall/F1), confusion matrix heatmap, and the "Generate AI Retention Strategy" button.
5. **Agent Report**: Full LangGraph pipeline output displaying risk level badge, risk summary, key risk drivers list, numbered recommended actions, agent reasoning, knowledge base sources, and ethical disclaimer.

5.2 Deployment

The application is hosted on **Streamlit Community Cloud** at `genaicapstoneproject.streamlit.app` The deployment is directly linked to the GitHub repository, enabling continuous deployment on code pushes. The Chroma vector store is committed to the repository (`chroma_db/`), eliminating the need for rebuilding embeddings at deployment time. The Groq API key is managed via Streamlit Secrets (not hardcoded). Python version is pinned to 3.12 for dependency compatibility.

6 Technology Stack

Table 3: Complete Technology Stack — Milestone 1 & 2

Component	Technology	Purpose
Milestone 1 — Machine Learning		
ML Framework	scikit-learn 1.6.1	Model training, evaluation, preprocessing
Imbalance	imbalanced-learn (SMOTE)	Training set oversampling
Data	pandas, NumPy	EDA, preprocessing, feature engineering
Serialisation	joblib	Model and scaler persistence (.pkl)
Visualisation	Plotly, Matplotlib	Charts, confusion matrix, gauge
Milestone 2 — Agentic AI		
Agent Framework	LangGraph	StateGraph workflow orchestration
LLM	Groq API (Llama 3.1 8B)	Intervention planning, JSON generation
RAG	Chroma + LangChain	Vector store and retrieval
Embeddings	intfloat/e5-small	Chunk embedding via sentence-transformers
Document Load	PyPDFLoader (LangChain)	Knowledge base PDF ingestion
LLM Integration	langchain-groq	Groq API client
Platform		
UI Framework	Streamlit	Interactive web application
Hosting	Streamlit Community Cloud	Public deployment
Version Control	Git / GitHub	Code management, CI/CD
Dataset	Telco Customer Churn	7,043 customers, 19 features (Kaggle)

7 Project Structure

```
genAI_capstone_project/
|-- app.py                        # 5-page Streamlit application
|-- requirements.txt              # Pinned Python dependencies
|-- .env.example                  # Environment variable template
|-- README.md                     # Setup and deployment guide
|-- data/
|   +-- Telco-Customer-Churn.csv  # 7,043 customer records
|-- models/
|   |-- logistic_regression_model.pkl
|   |-- decision_tree_model.pkl
|   |-- random_forest_model.pkl
|   |-- scaler.pkl
|   |-- model_columns.pkl
|   |-- test_data.pkl             # Pre-saved test split for live
   metrics
|   +-- metrics.json              # Pre-computed evaluation scores
|-- knowledge_base/               # 7 industry retention research PDFs
|-- chroma_db/                    # Pre-built Chroma vector store (
   committed)
```

```
|-- notebook/
|   +-- Telco_Customer_Churn.ipynb  # EDA, training, evaluation notebook
|-- src/
|   |-- preprocessing.py          # Input preprocessing for inference
|   |-- model_training.py         # Model loader utilities
|   |-- evaluation.py             # ROC curve and confusion matrix
   helpers
|   +-- agent/
|       |-- state.py              # ChurnAgentState TypedDict definition
|       |-- nodes.py              # Four LangGraph node functions
|       |-- graph.py              # StateGraph compilation
|       |-- prompts.py            # LLM prompt templates
|       |-- retriever.py          # Chroma similarity retrieval
|       |-- embedder.py           # Vector store builder
|       |-- document_loader.py    # PDF chunker
|       |-- build_vectorstore.py  # One-time vector store builder script
|       +-- test_agent.py         # End-to-end integration test
+-- Documentation/
    +-- Report.pdf
```

Listing 5: Repository Structure

8 Ethical Considerations

8.1 Bias and Fairness

The model uses demographic attributes including `gender`, `SeniorCitizen`, and `Partner` status. While included for predictive completeness, their use in real-world deployment raises fairness concerns. Organisations deploying this system should conduct fairness audits (e.g., disparate impact analysis across demographic groups) before using model outputs to drive customer decisions.

8.2 Data Privacy

The dataset is publicly available and anonymised. In a production context, all customer data must be handled in compliance with applicable regulations (GDPR, India's DPDPA, CCPA). The `customerID` field was removed from the modelling pipeline to prevent indirect identification. No real customer data was used at any stage.

8.3 Model Transparency

All three Milestone 1 models provide varying levels of interpretability. Decision Trees offer explicit rule-based explanations suitable for regulatory compliance. Logistic Regression coefficients directly encode feature contributions. Random Forest feature importances, while aggregate, support high-level business explanations. The application exposes these importance scores to end users.

8.4 AI-Generated Recommendations

All Milestone 2 retention recommendations are accompanied by an explicit disclaimer (reproduced in Section 4.5) clarifying that outputs are decision-support tools for human agents, not autonomous decision-making systems. This directly addresses the Ethical AI criterion in the evaluation rubric.

8.5 Responsible Use

Churn predictions are probabilistic estimates and must not be treated as definitive. Retention interventions driven by these predictions should be reviewed by qualified human agents to avoid penalising low-risk customers or misallocating retention budgets.

9 Limitations and Future Work

9.1 Current Limitations

- The Telco dataset, while realistic, is synthetic and may not fully capture real-world telecom customer behaviour.
- Model recall remains moderate (∼67–72%); high-recall production use cases may require threshold tuning or more advanced ensemble methods such as XGBoost.
- The Groq free-tier API introduces latency variability under high load.
- The knowledge base is static; real-world deployment would benefit from a continuously updated retrieval corpus.
- The `intfloat/e5-small` embedding model is compact for deployment efficiency but a larger model (e.g., `e5-large`) would likely improve retrieval relevance.

9.2 Future Enhancements

- **Dynamic knowledge base**: Automated ingestion of new retention research, case studies, and company-specific playbooks into the vector store.
- **Multi-customer batch processing**: Extend the agent to process bulk CSV uploads and generate population-level retention dashboards.
- **Feedback loop**: Capture retention team feedback on recommendations to fine-tune prompts and retrieval over time.
- **Structured output validation**: Use Pydantic's `BaseModel` with LangChain's `.with_structured_out` for schema-enforced LLM outputs rather than JSON string parsing.
- **A/B testing framework**: Evaluate which recommended actions lead to measurable improvements in real retention outcomes.

10 Conclusion

This report documents the complete two-milestone AI-driven customer churn prediction and retention strategy system. Milestone 1 successfully implements a rigorous classical ML pipeline — covering preprocessing, SMOTE balancing, training of three supervised models, and deployment of an interactive Streamlit application. The Random Forest classifier achieves the strongest performance (F1: 64.39%), while Logistic Regression and Decision Tree provide complementary interpretability benefits. Feature importance analysis surfaces actionable business insights around contract type, tenure, and monthly charges.

Milestone 2 extends this foundation into a fully agentic AI system. The LangGraph-orchestrated pipeline autonomously assesses churn risk, retrieves evidence-grounded re-

tention strategies from a seven-document knowledge base via RAG, plans personalised interventions using Groq-hosted Llama 3.1 8B, and generates structured retention reports with cited sources and ethical disclosures. The complete system is publicly deployed and demonstrates full-stack AI engineering from predictive machine learning to autonomous agentic reasoning.

References

[1] Blastchar. *Telco Customer Churn Dataset*. Kaggle, 2018. https://www.kaggle.com/datasets/blastchar/telco-customer-churn

[2] Pedregosa, F. et al. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[3] Chawla, N.V., Bowyer, K.W., Hall, L.O., Kegelmeyer, W.P. SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16:321–357, 2002.

[4] Breiman, L. Random Forests. *Machine Learning*, 45(1):5–32, 2001.

[5] LangChain AI. *LangGraph Documentation*. https://langchain-ai.github.io/langgraph/, 2024.

[6] Groq Inc. *Groq API Documentation*. https://console.groq.com, 2024.

[7] Chroma. *Chroma Vector Database Documentation*. https://docs.trychroma.com, 2024.

[8] Streamlit Inc. *Streamlit Documentation*. https://docs.streamlit.io, 2024.

[9] Wang, L. et al. Text Embeddings by Weakly-Supervised Contrastive Pre-training. *arXiv preprint arXiv:2212.03533*, 2022.

Live Application: genaicapstoneproject.streamlit.app
Repository: github.com/Shreyashgol/genAI_capstone_project

Submitted for the End-Semester Evaluation — GenAI Capstone Course, 2026